Technische Universität Berlin

Distributed Systems
Chapter 3: Architecture Models

FG Verteilte Systeme und Betriebssysteme | Distributed Systems

Agenda

3.1 Client / Server Systems

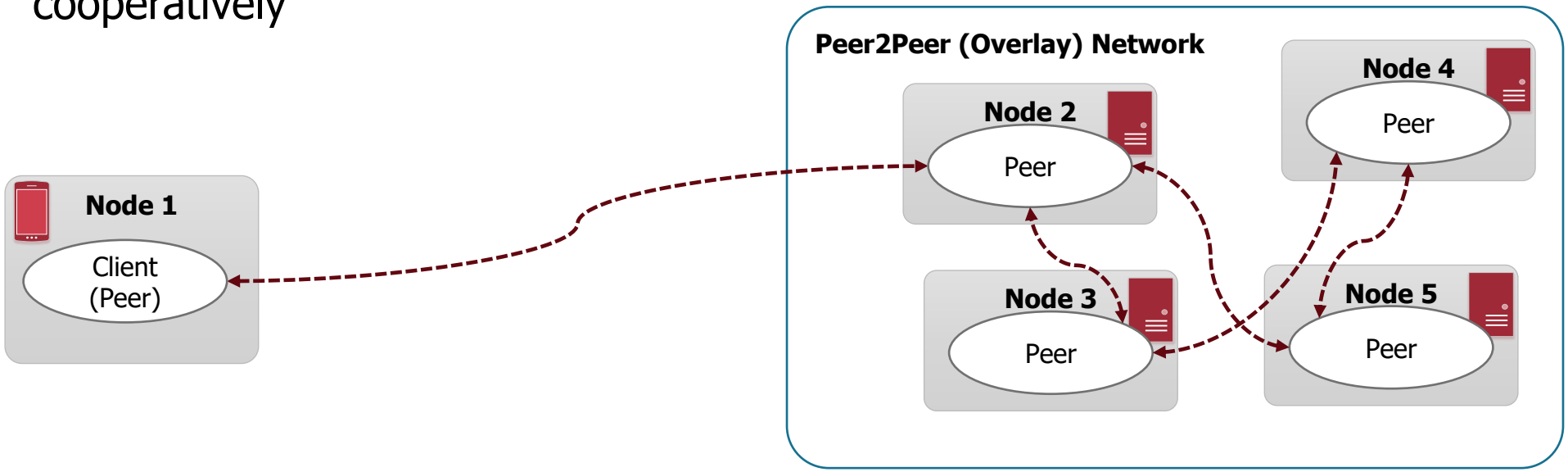
3.2 Peer-to-Peer Systems

3.3 Publish / Subscribe Systems

Peer-to-Peer Systems

Peer-to-peer systems consist of equal nodes without a central entity

- No separation of roles and responsibilities
- All entities (peers) run the same program, offer the same interfaces and interact cooperatively



Peer-to-Peer Systems

Peer-to-peer systems consist of equal nodes without a central entity

- No separation of roles and responsibilities
- All entities (peers) run the same program, offer the same interfaces and interact cooperatively

Necessary capabilities

- Look-up mechanisms: Where is the appropriate data located?
- Decentralized storage and transmission of requested data

Peer-to-Peer Systems – Properties

No sharp definition but principle of organization

- Every node performs the same types of tasks, no distinction between client and server

Core concepts (sometimes not all of them apply)

- Self organization, no central management
- Sharing of resources: Use of resources at the "edge" of the network
- Nodes come and go, peers are autonomous (make own decisions about offers)
- Addressing on application level
- Look-up mechanisms: Where is the appropriate data located?
- Decentralized storage and transmission of requested data
- Large number of peers

Peer-to-Peer Systems – Pros and Cons

Advantages

- Better availability/reliability: no single point of failure
- Up-to-date data / easy to publish data
- Access to unpublished data, anonymity is preserved
- Scalability
- Shared resources (computing, storage, bandwidth)
- No censoring/manipulation by central entity

Disadvantages

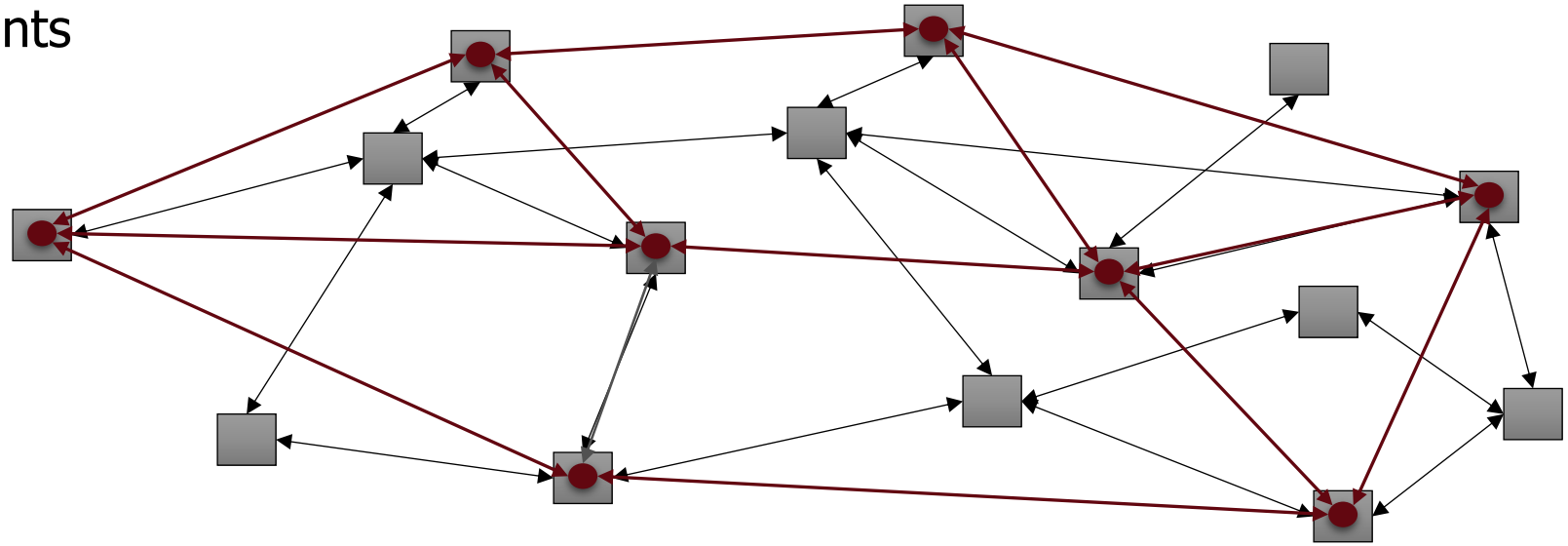
- Complex development and administration
  - Programming, debugging
  - Security and routing / searching
  - Replication
- No reliability („just" probabilistic) guarantees
- Trustworthiness of participating peers?

Example Peer-to-Peer System – Gnutella

Gnutella does not require a central server

Gnutella is a so-called overlay network (over the internet)

- A virtual topology established by the application
- Peers are called servants



Peer-to-Peer Systems – Bootstrapping

A fully decentralized architecture raises some questions:

- Bootstrapping and
- finding data

Bootstrapping: How to establish the first connection (with which node)?:

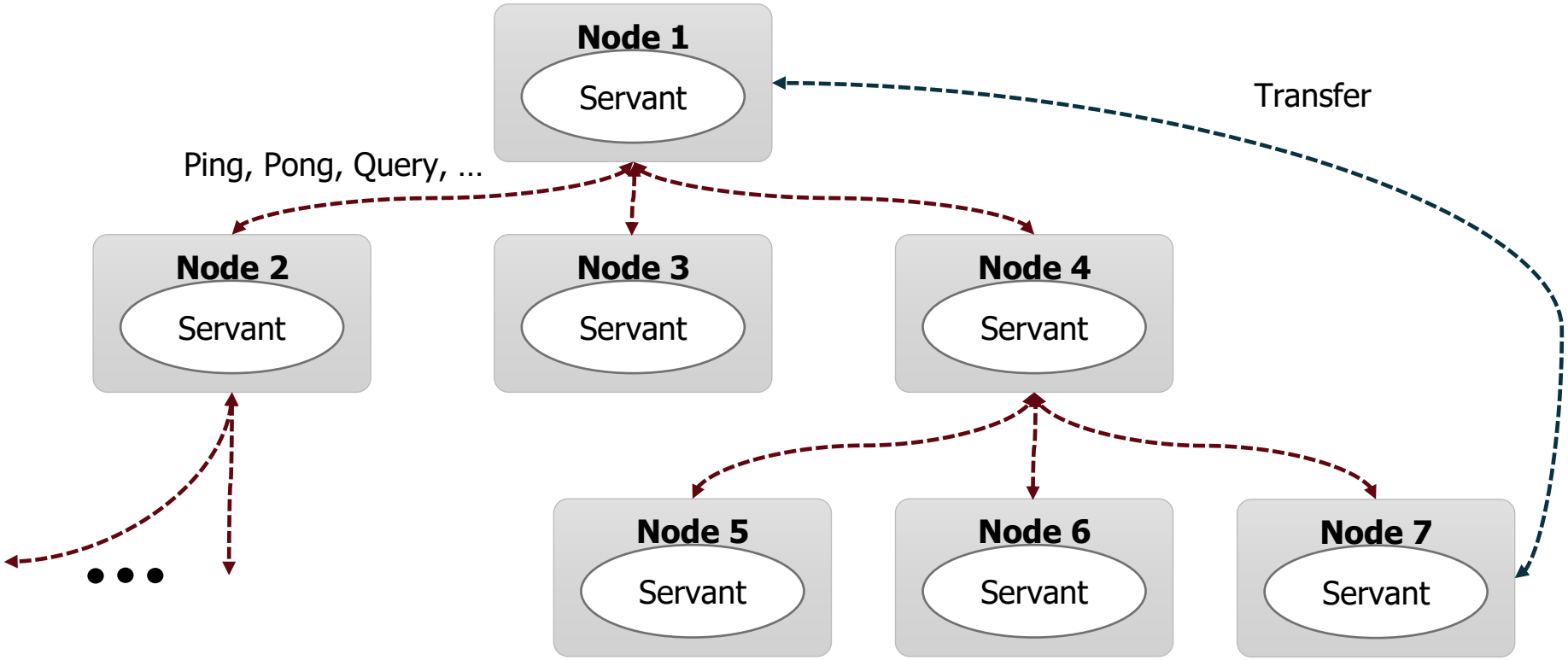
- Generally known nodes (published over web server)
- List with all known nodes from past sessions
- Discover further nodes using start node

Peer-to-Peer Systems – Finding data

A fully decentralized architecture raises some questions: How to find data?

- Flooding: broadcast search to all nodes
- Nodes remember IDs of already recognized messages, thus preventing loops
- TTL (Time to Live) limits lifetime of a message
- Quality of nodes can be measured using different criteria
  - Connection
  - Utilization
  - Quality of the file

Peer-to-Peer Systems – Gnutella Protocol



Peer-to-Peer Systems – Gnutella Protocol

Gnutella Protocol uses a set of distinct message to interact with other peers

- First message: CONNECT with known peer (is confirmed with OK)
- Further communication using distinct messages (see descriptor). Rules:
  - Forwarding descriptor (message) → increase hop count by 1
  - If Hops = TTL (Time To Live) → message is discarded (default for TTL: 7)
  - If message has identical IDs and identical payload compared to previous one → reject

Field	Bytes	Description
ID	16	Random number to distinguish messages
Descriptor	1	Kind of message: Ping, Pong, Query, Query_Hit, Push
TTL	1	Time To Live (maximum number of forwardings)
Hops	1	Number of already applied forwardings
Length	3	Length of „Payloads“

Peer-to-Peer Systems – Gnutella Protocol

Why does Gnutella use seven hops as a default for the TTL?

Milgram's Experiment (1967): Social experiment with people - who knows whom?

- Letters to a friend of Milgram were sent to random people in Nebraska
- "Routing-Information":
  - Name of target person
  - place (“Boston”)
  - occupation (“stock broker”)
- Instruction: only mail to people you know personally on a first name basis

Result: on average 6 steps to reach target person!

Agenda

3.1 Client / Server Systems

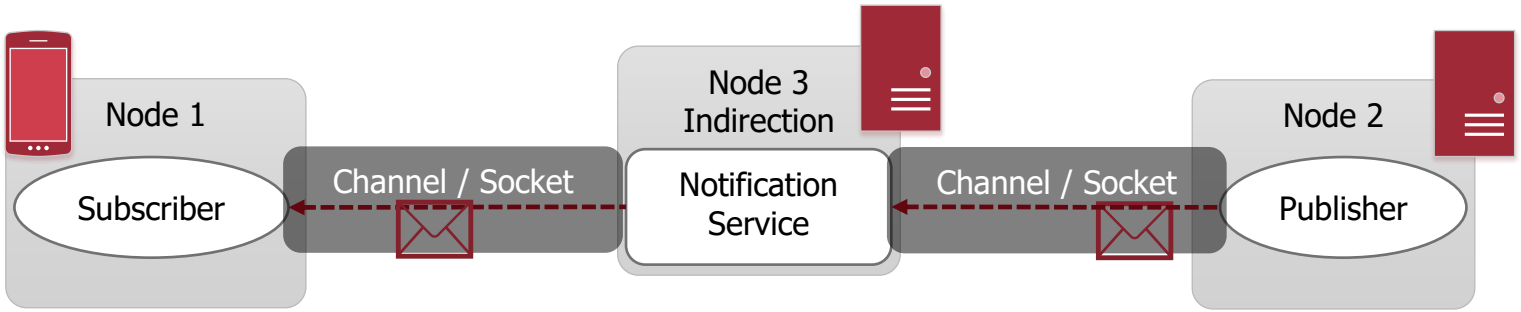
3.2 Peer-to-Peer Systems

3.3 Publish / Subscribe Systems

Publish / Subscribe Systems

Remember Architectural Model – Communication Paradigm

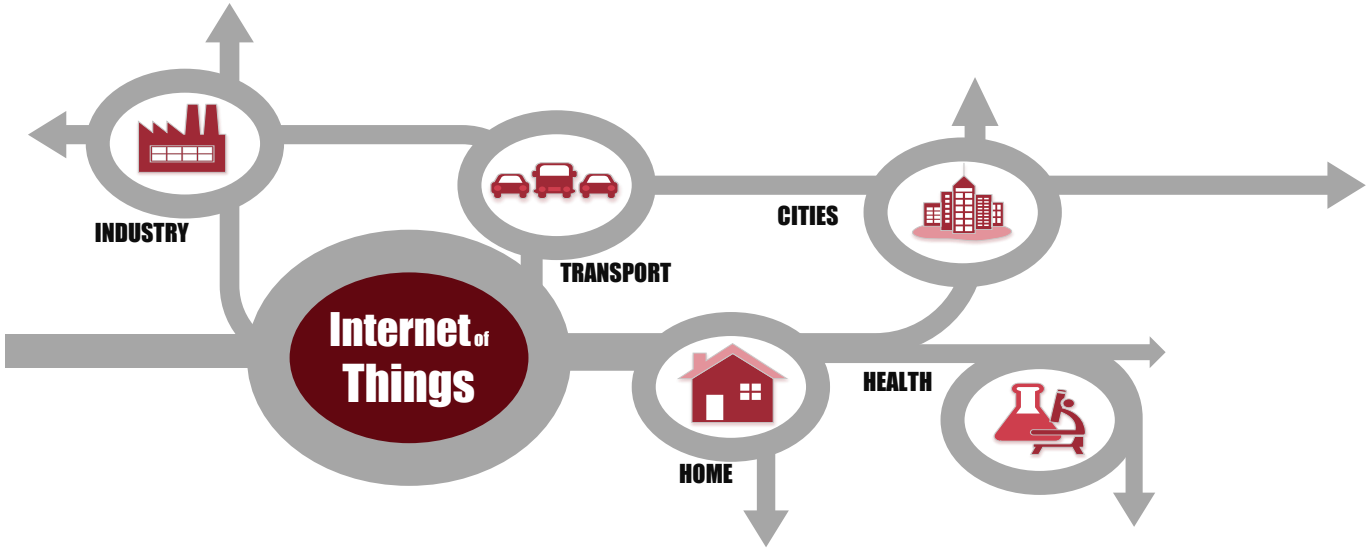
- Publish / subscribe is a variant of the indirect communication paradigm
- Due to its capabilities that go beyond simple message passing, it is sometimes referred to as a whole architectural model



Publish / Subscribe Systems - Motivation

With the Internet of Things and its applications (e.g. smart cities), a trend towards information-driven applications can be observed

- Data sources that generate data based on external events/automatically
- Applications select, filter and combine incoming streams of data and react to changes



Publish / Subscribe Systems - Motivation

Information no longer static, rather flows from producers to consumers

- Mechanisms to access static data (queries) different from those controlling the flow of information (subscriptions/filters)

Application examples:

- Personalized news service
- Stock monitoring software
- Distributed auction platform
- Wireless Sensor Networks
- Near-Real-Time, IoT applications

Publish / Subscribe vs. Client / Server Systems

Client/server systems well suited for static information

- Clients request data or functionality from a server, server reply to clients' requests
- Synchronous communication (initiated by the client),
- Direct addressing → tight coupling

Disadvantages regarding information-driven applications

- Periodic polling of data source
- Communication regardless whether new data is available
- Every client queries individually

Publish / Subscribe vs. Client / Server Systems

Publish/subscribe systems: Two roles

- Producers publish notifications
- Consumers subscribe to notifications

Notification Service (NS) delivers notifications

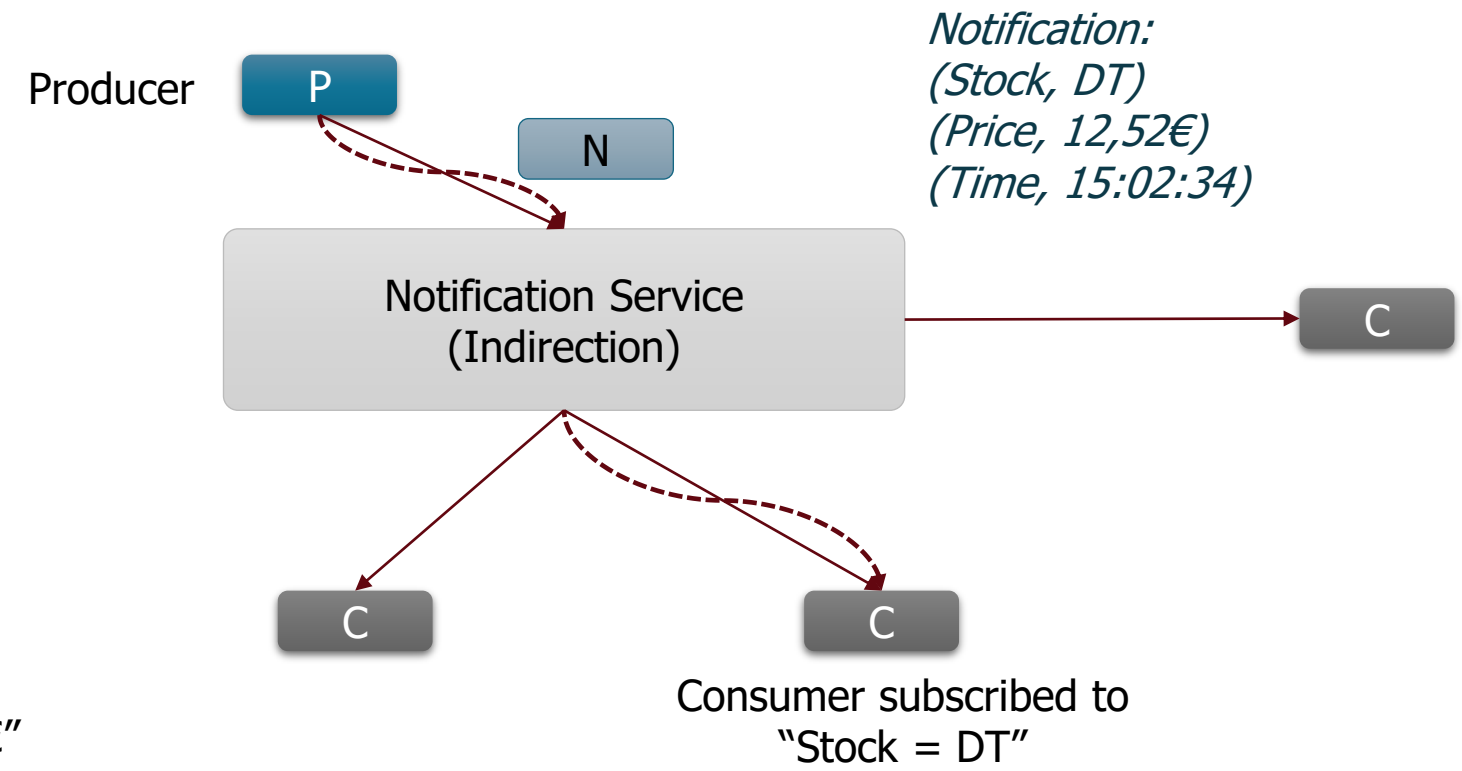
Characteristics

- Asynchronous communication (initiated by producer)
- Indirect addressing (anonymous) → loose coupling

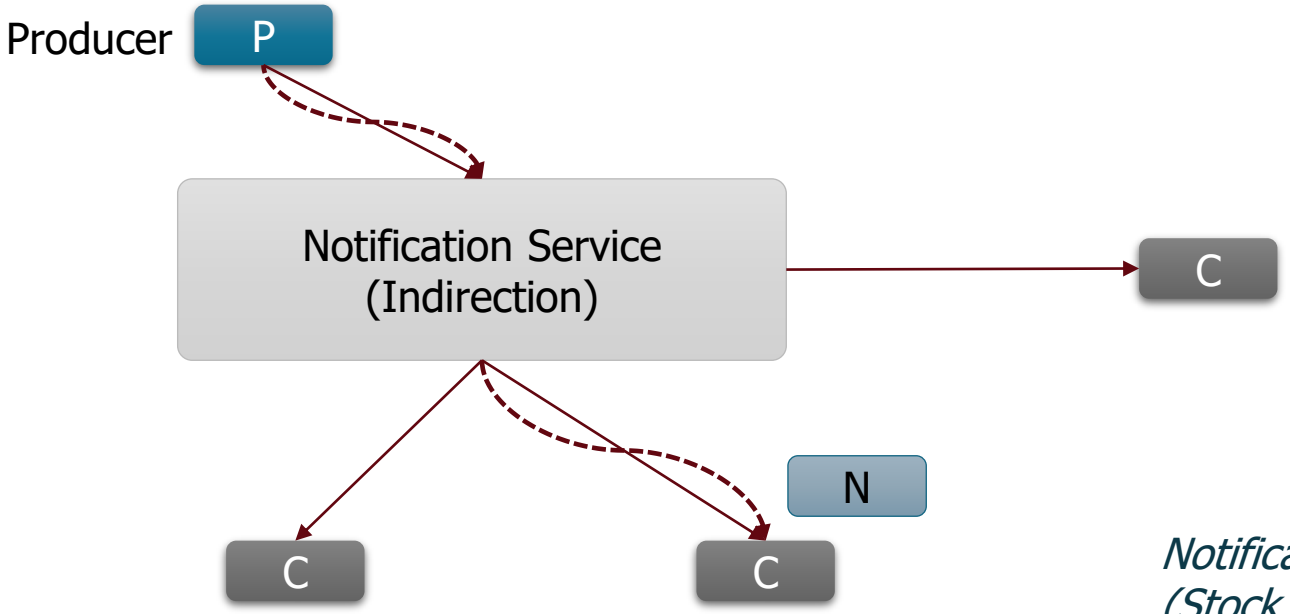
Advantages regarding information-driven applications

- Communication only occurs when new data is available
- Low latency because initiation done by producer
- Distribution to many consumers enables optimization

Publish / Subscribe vs. Client / Server Systems



Publish / Subscribe vs. Client / Server Systems



Consumer subscribed to
"Stock = DT & Price > 13€"

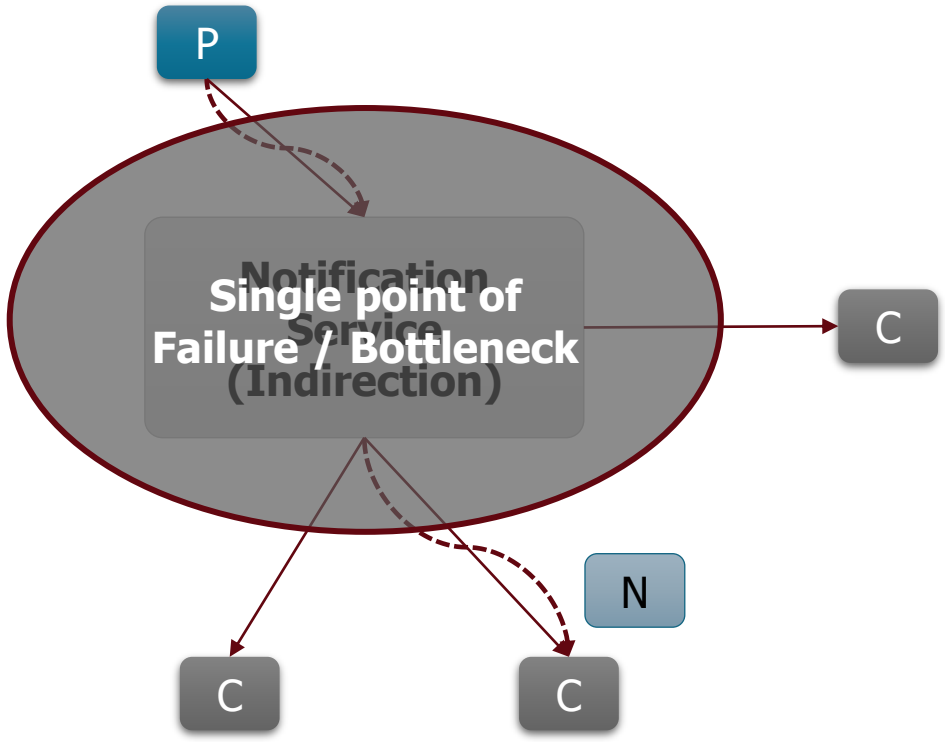
Consumer subscribed to
"Stock = DT"

Notification:
(Stock, DT)
(Price, 12,52€)
(Time, 15:02:34)

Publish / Subscribe Systems

Important aspects of pub/sub systems:

- How to select messages of interest? → notification selection
- How to deliver notifications to interested consumers?
  - What is the problem with the previous model of our system?
  - Centralized approach for the Notification Service (NS) does not scale
  - Notification routing is required if a distributed approach for the NS is used



Publish / Subscribe Systems – Notification Selection

Expressiveness of notification selection is crucial for a notification service

- Trade-Off: Expressiveness vs. Scalability
- Common Approaches:
  - Channels
  - Subjects
  - Event Types
  - Content-based Filtering

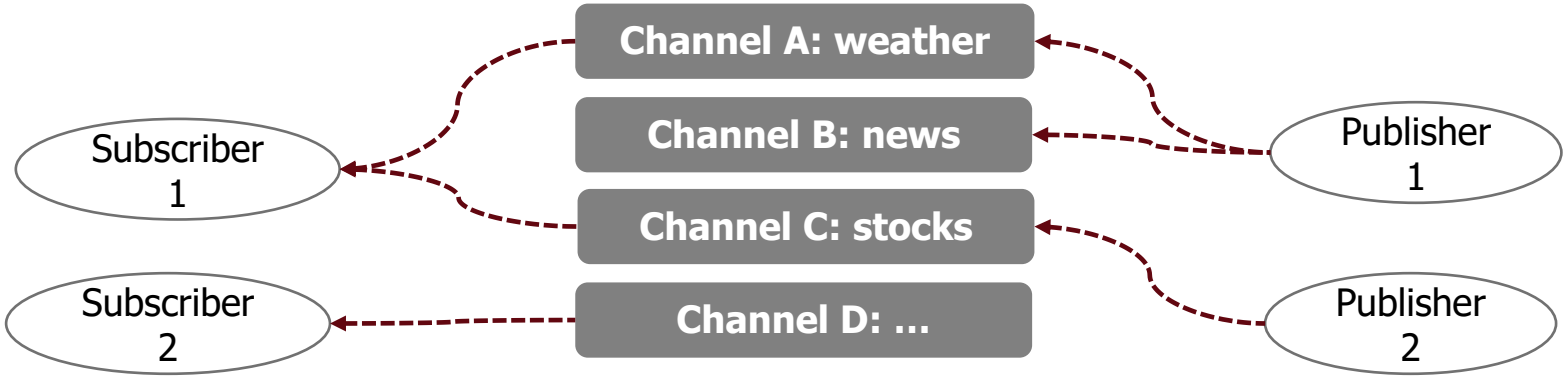
Increasing Expressiveness

Publish / Subscribe Systems – Notification Selection

Channels provide a rather primitive approach for selection without using filters

Producer publishes notification with respect to a channel

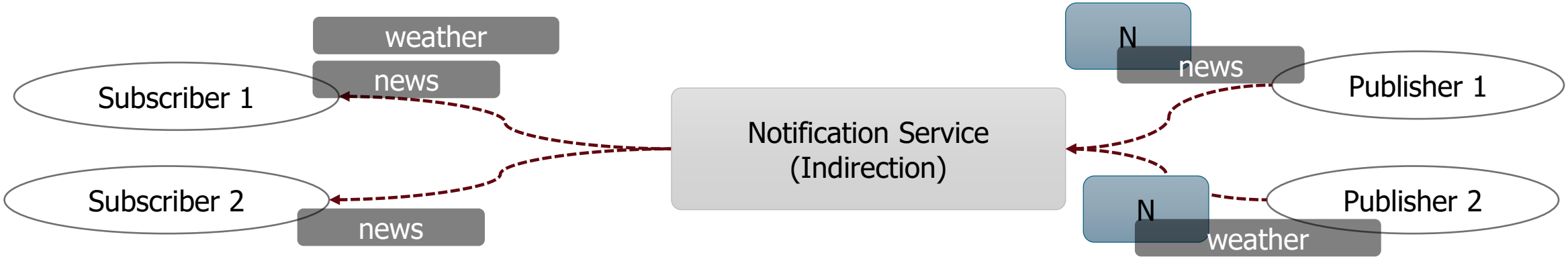
- Producer determines channel → Producer and consumer not fully decoupled
- Easily mappable to IP-multicast
- Used by CORBA-Event service



Publish / Subscribe Systems – Notification Selection

Topics or subjects decouple notifications from channels

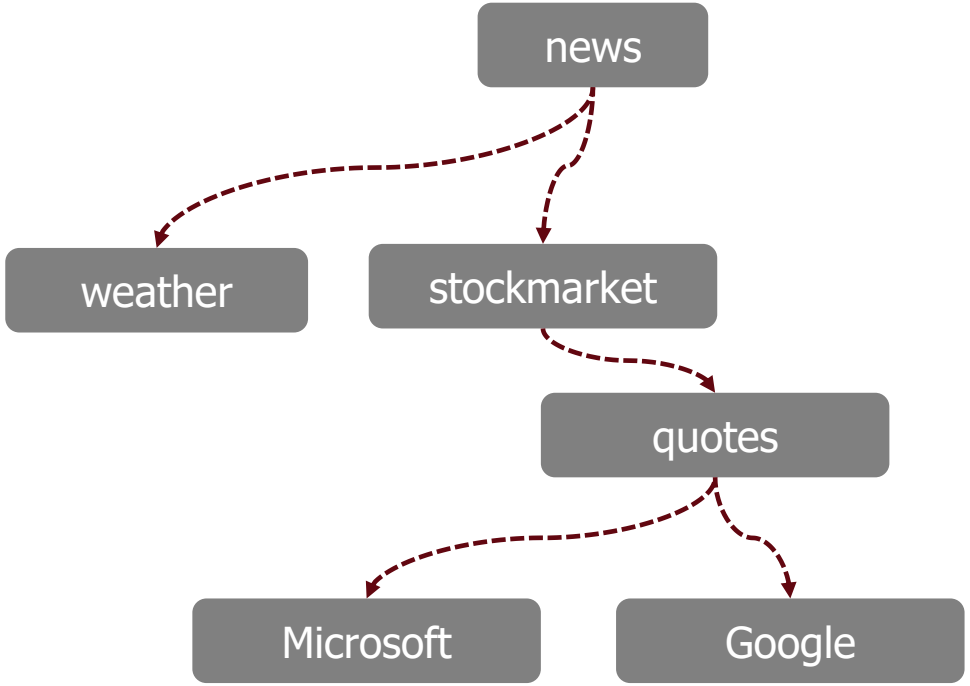
- Notifications carry a topic identifier



Publish / Subscribe Systems – Notification Selection

Topics or subjects decouple notifications from channels

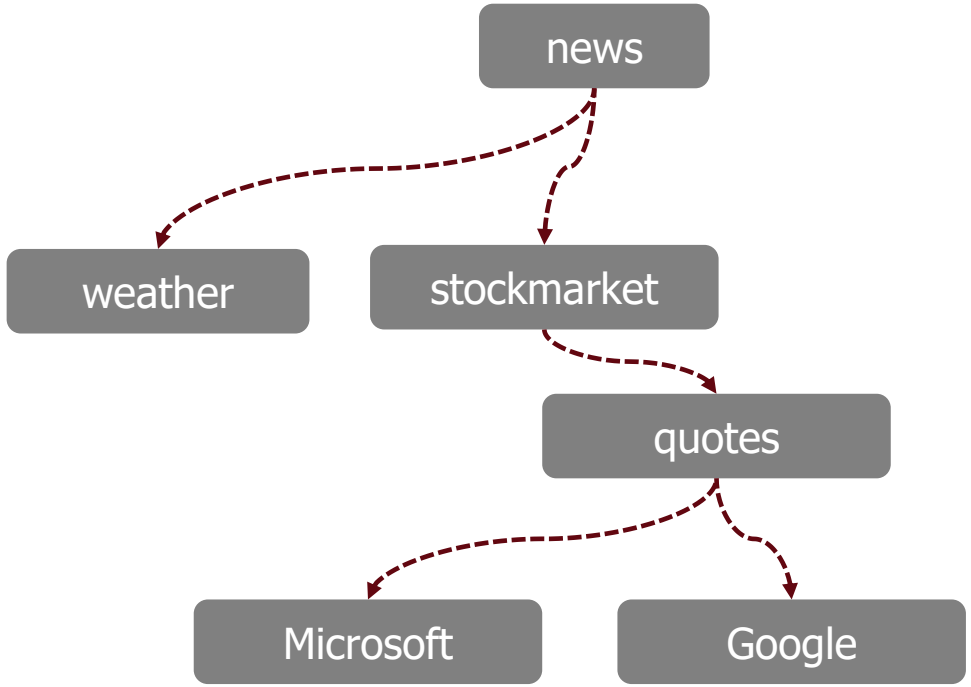
- Notifications carry a topic identifier
- Topics can be organized hierarchically → increased expressiveness
- Wildcards allow to select whole sub-trees, e.g.: news.stockmarket.quotes.*



Publish / Subscribe Systems – Notification Selection

Problem: How to divide the notification space into subjects?

- Division has implications on possible selections
  - news.stockmarket.quotes.microsoft or news.stockmarket.microsoft.quotes ?
- Subjects only suited for categorizing along a single dimension
- Changing subject tree has implications on consumers



Publish / Subscribe Systems – Notification Selection

Event types (type-based selection) are comparable to object-oriented programming approach

- Notifications are categorized based on types
- Usually, a common abstract superclass event exists
  - Event is subclassed to generate specific classes of events (e.g. stockquote)
  - Consumers subscribe to classes of events
  - Subclasses are also delivered
  - Filters can be based on type names or include attributed and methods of objects, similar to content-based selection
  - Single- and Multi-Inheritance possible

Publish / Subscribe Systems – Notification Selection

Compared to topic-based selection, content-based allows filtering using a range of fields in the notification

- Based on a data model with corresponding filter model, e.g. name/value pairs with conjunctive attribute filters
- Filters evaluate the whole content of notifications using logical expressions comprised of filter predicates, e.g. {(Stock = Microsoft & Price > 15€)}
- Filter F is a function that maps notifications to Boolean values true and false
- Set of matching notifications N(F) = {n | F(n) = true}

Publish / Subscribe Systems – Broker Network

A centralized implementation of the Notification Service has several drawbacks:

- Single point of failure
- Bottleneck – bandwidth and computing power
- No exploitation of locality although most interactions will be local
- Neither reliable nor scalable to wide-area scenarios

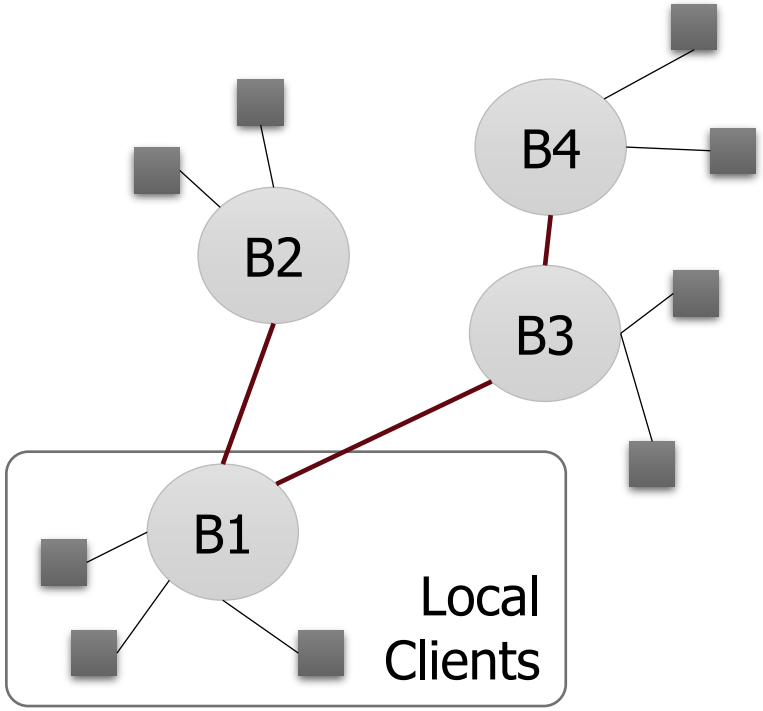
A distributed implementation of the Notification Service requires:

- Replication / Partitioning
- Cooperation between the instances of the service

Publish / Subscribe Systems – Broker Network

Given a distributed implementation, the notification services are called brokers

- Each broker manages a set of local clients to which it forwards matching notifications
- Notifications are stepwise forwarded through broker network
- Cyclic forwarding and delivery of duplicates must be avoided
  - Easy in acyclic topologies → fault tolerance?
  - Complex in topologies with cycles



Publish / Subscribe Systems – Broker Network

How to route notifications through the network?

Flooding delivers each notification to all brokers

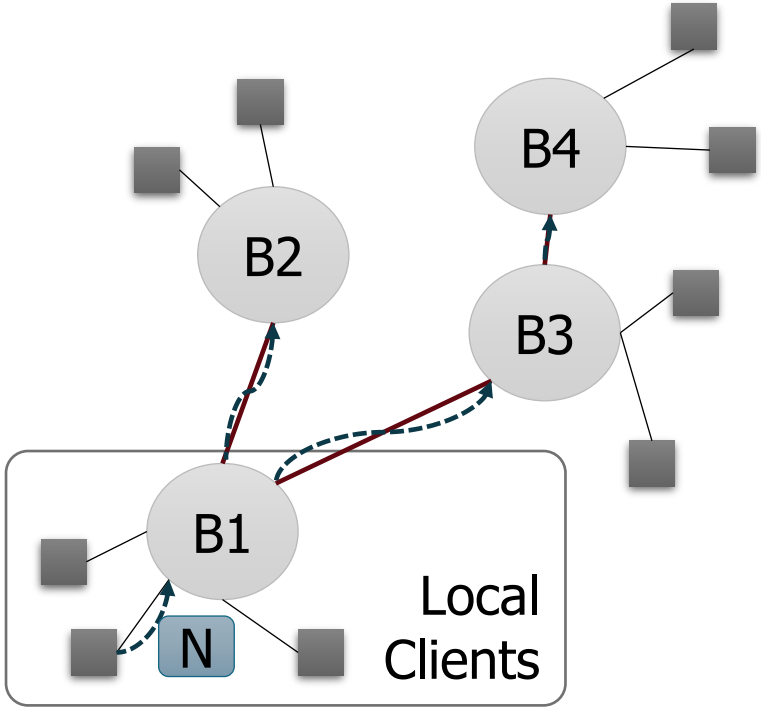
- A broker forwards an incoming notification to all neighbors if it was received from a local client and to all other neighbors if it was received from a neighbor

Advantages

- No routing tables for neighbor brokers necessary
- Subscriptions become active immediately
- Implementations can easily exploit IP-Multicast

Disadvantage

- Unnecessary overhead due to poor forwarding method



Publish / Subscribe Systems – Broker Network

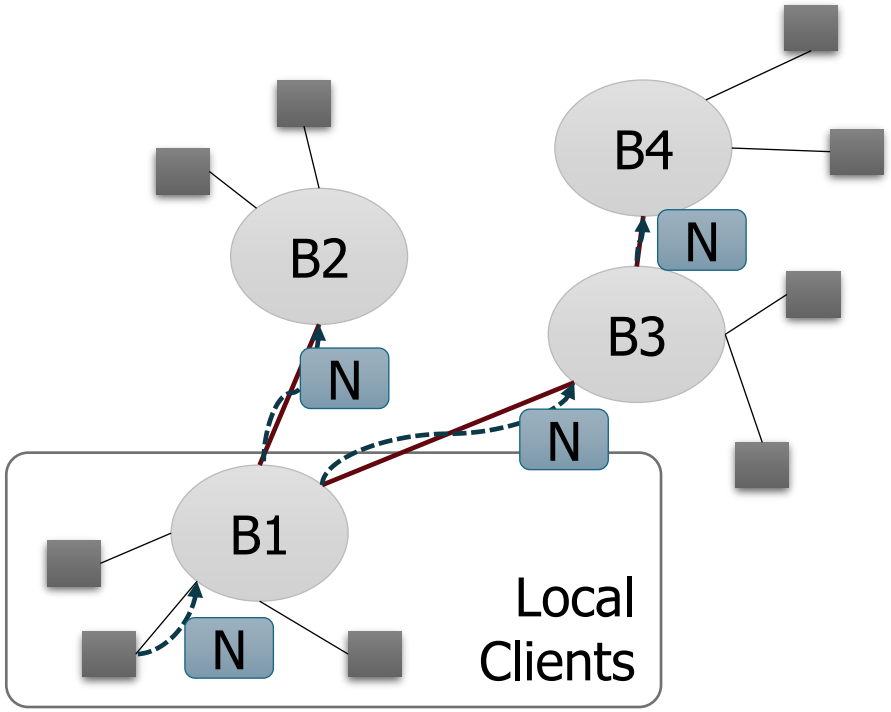
Flooding delivers each notification to all brokers

Efficient if:

- Most notifications need to be delivered to a majority of brokers
- Subscriptions are highly transient

Not efficient if:

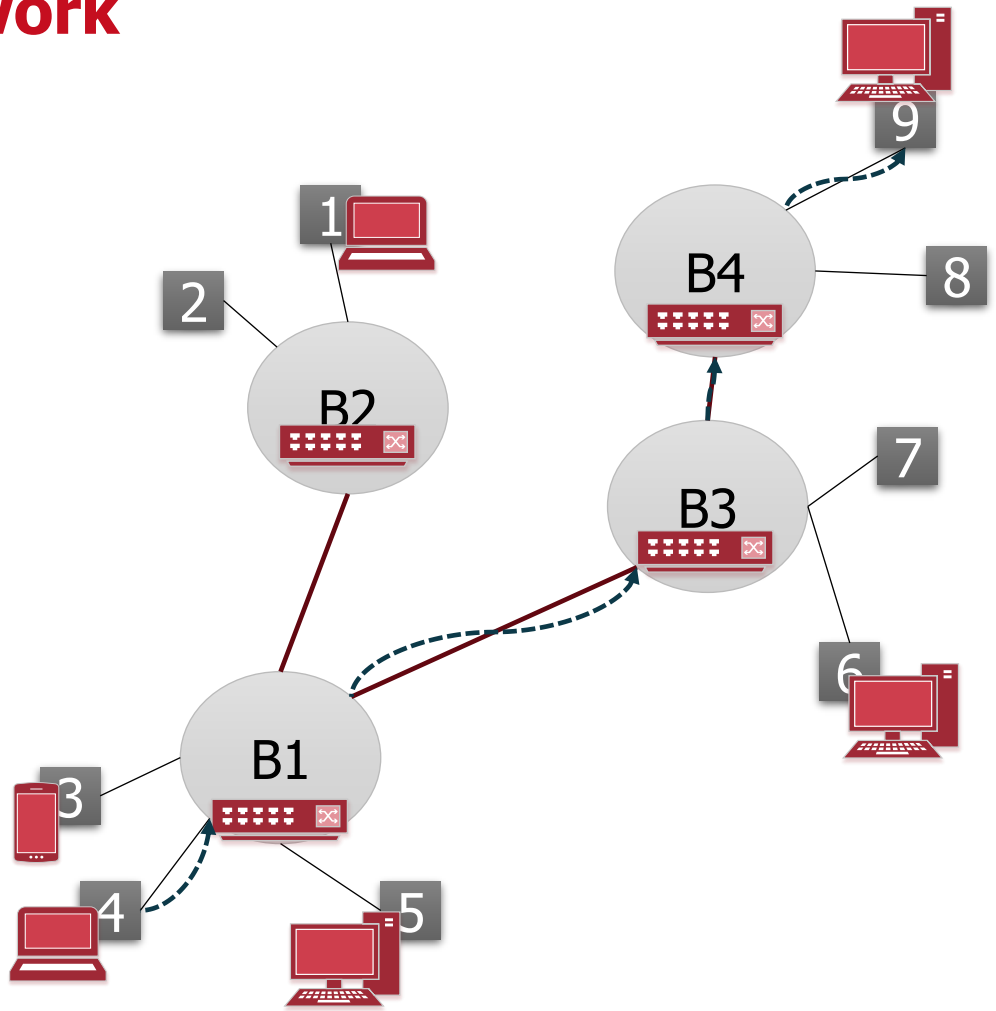
- Filters are highly selective
- Locality among subscriptions exist



Publish / Subscribe Systems – Broker Network

How can we improve the Flooding approach?

- Think of IP-based networks, Flooding is similar to broadcasting each message
- Given a switched network, how do IP packets reach their destination?
- Introduce a routing approach: Use filters to forward notifications only if required

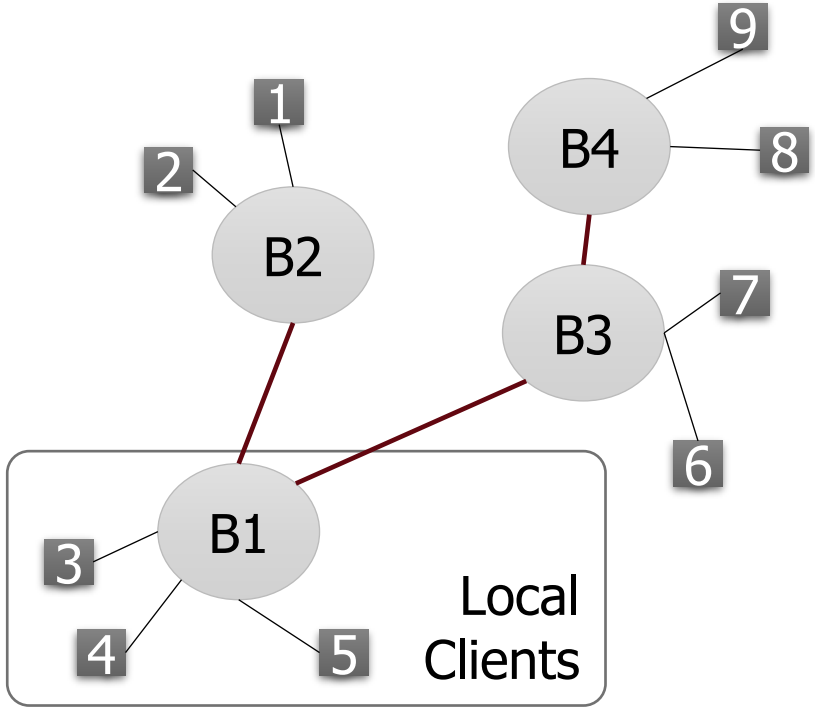


Publish / Subscribe Systems – Broker Network

Content-based (also filter-based) routing only forwards notifications through the network when there is a valid path to a subscriber

Each broker maintains:

- Neighbor List: Contains all neighbor brokers
- Subscription List: Contains all served clients
- Routing Table
  - Subscriptions are used as filters for notifications
  - The table consists of (F,D) entries
  - D is the destination, F is the filter
  - Matching notifications: N(F) = {n|F(n) = true}



Publish / Subscribe Systems – Broker Network

Content-based (also filter-based) routing only forward notifications through the network where there is a valid path to a subscriber

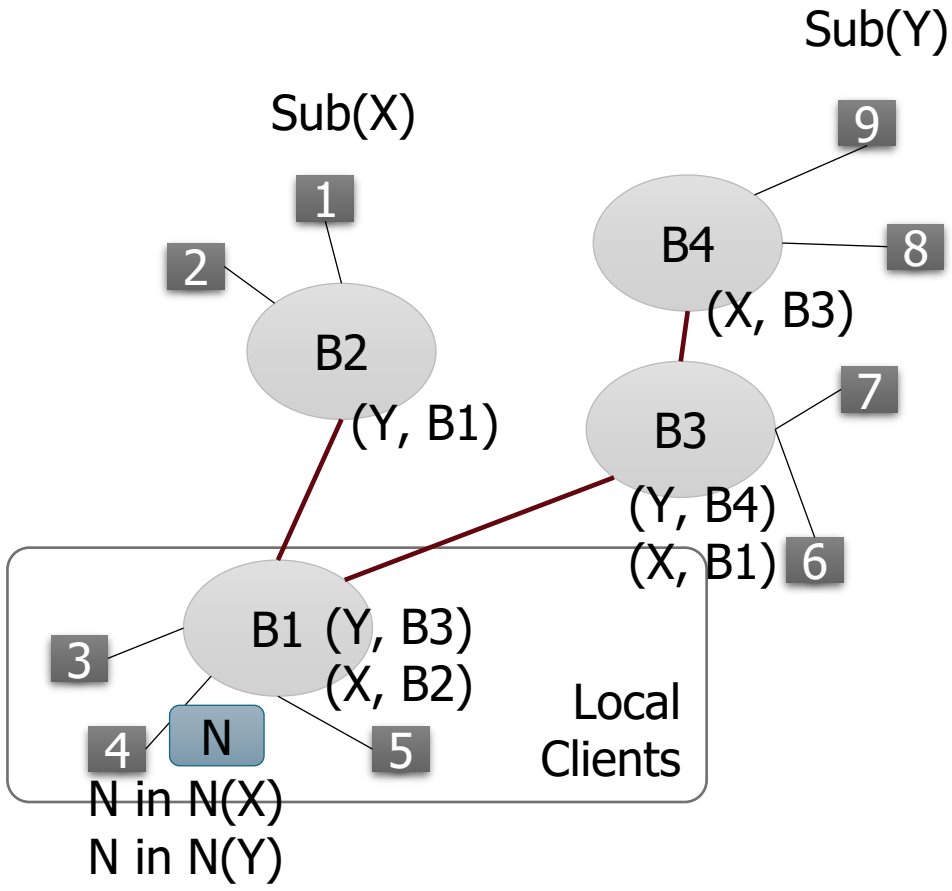
Simple Routing Algorithm:

```
Upon receive publish(notification n) from node x
   matchlist := match(n, subscriptions)
   send notify(n) to matchlist
   fwdlist := match(n, routing)
   send publish(n) to fwdlist - x
Upon receive subscribe(filter f) from node x
   if x is local client
      add x to subscriptions
   else
      add(f, x) to routing
   send subscribe(f) to neighbors - x
```

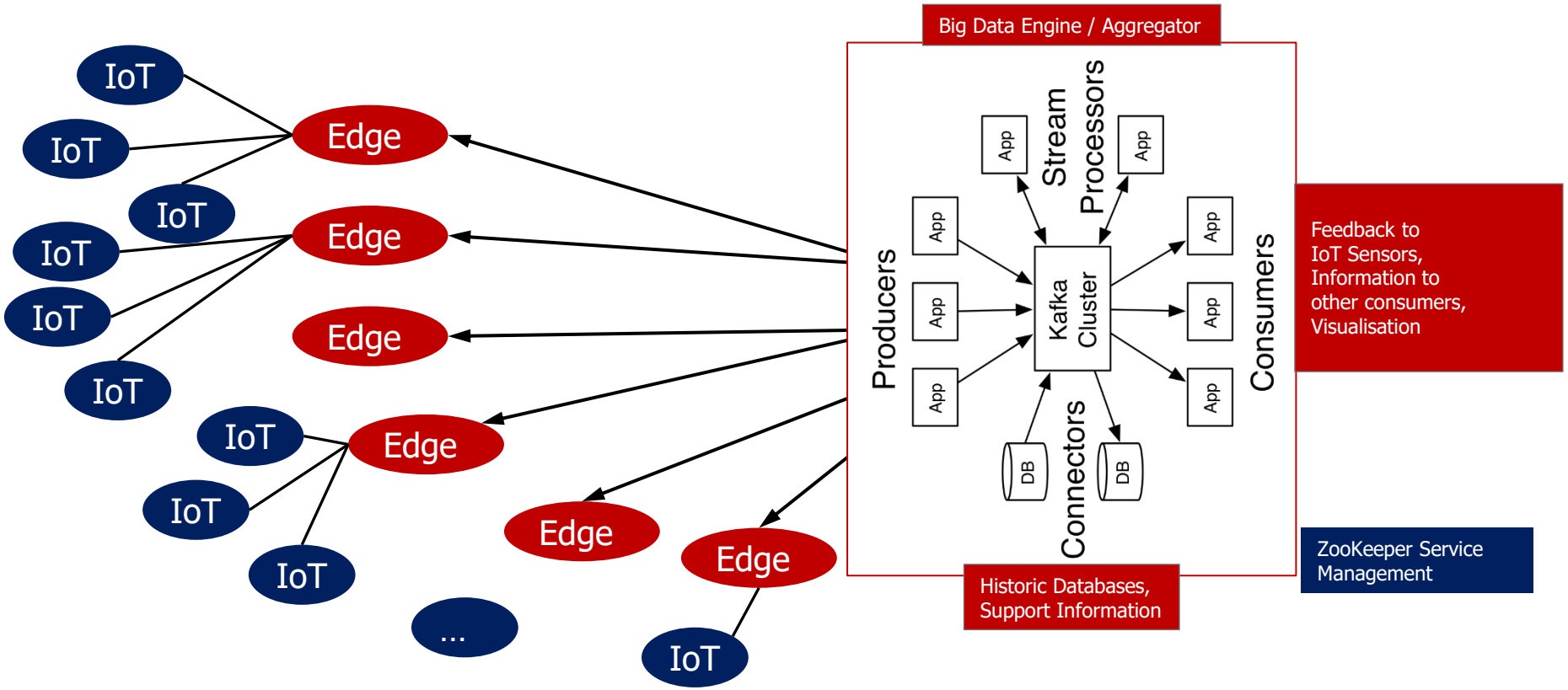
Publish / Subscribe Systems – Broker Network

Example

- Node 1 subscribes using filter X
- Node 9 subscribes using filter Y
- Node 4 publishes notifications that match X and Y
- Lists for B1:
  - Neighbor List: B2, B3
  - Subscription List: 3,4,5
  - Routing Table (RT): (Y, B3), (X, B2)



IoT and Role of Kafka: Make decision, reply



Kafka basic scenario

Three tasks in data analytics

- Select data to be processed: connect data sources
- Send the data to processing unit: Big data, monitoring, analysis, logging
- Store the results in final database or forward the decision to devices

Tasks in IoT environment

- Connect many, many heterogenous devices
- Receive the data and classify according priority, latency requirements, persistance
- Process the data and send the results to IoT

Kafka vs Traditional Messaging Systems (JMS, AMQ, RabbitMQ)

Kafka

- Scalable ⇒ Messaging system not scalable to Tbytes of data
- Replication, Partitioning ⇒ Fault tolerant data processing
- Distributed over network boundaries
- Simple to integrate, de-facto standard

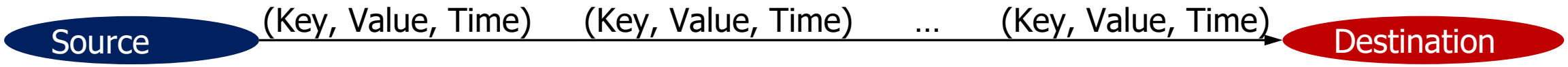
Relaxation on delivery semantics ⇒ no acks

- Kafka does not care if the Consumer consumed the message or not

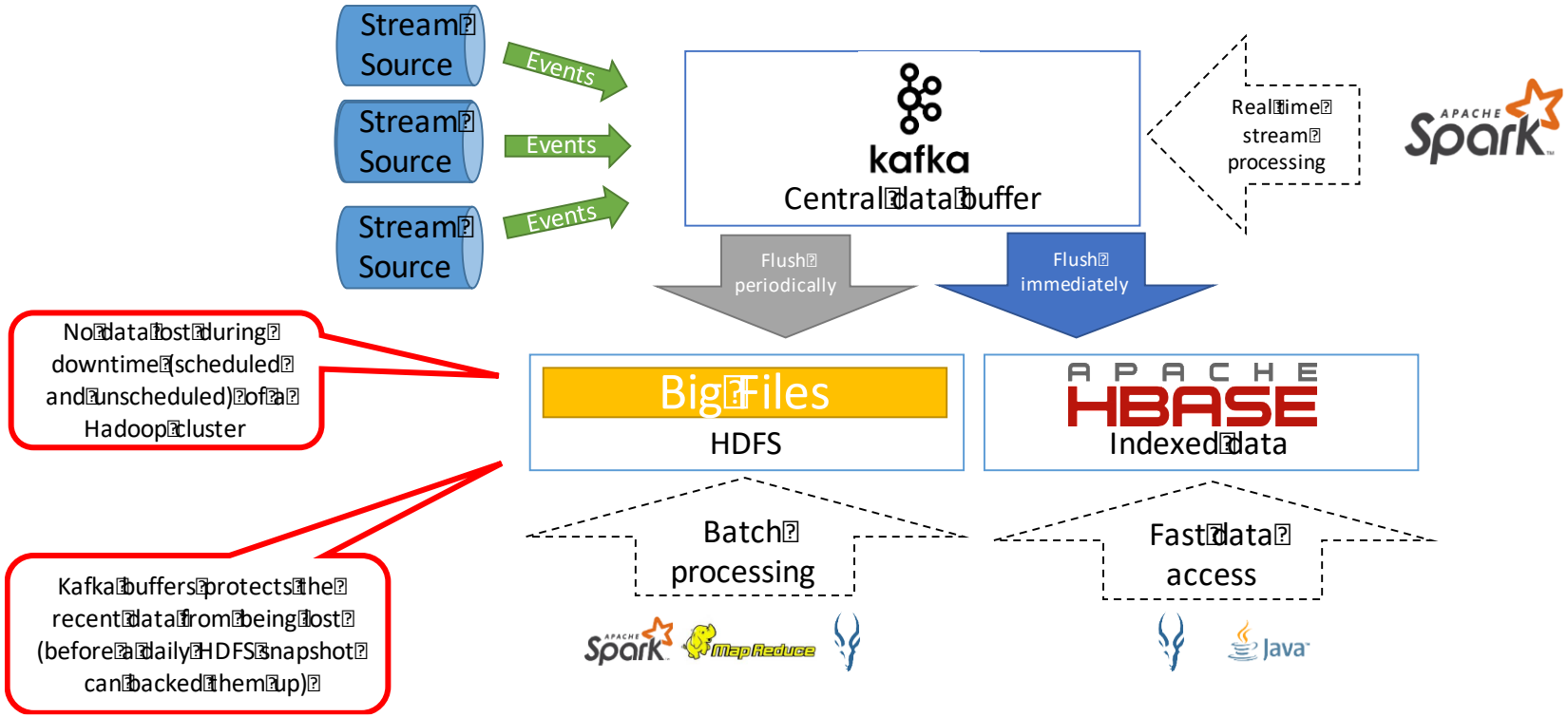
Basic Notion: Stream

What happens to the stream?

- Store and Forward to Data Analytics / Postprocessing
- Flush immediately and process with low latency



Kafka for Data Integration and Processing



Kafka as distributed streaming platform

Concepts

- Run on a cluster with one or more servers
- Each record consists of a key, a value, and a timestamp
- Kafka cluster stores streams of records in categories called topics

Modes of operation

- Publish /subscribe to streams of records (message queue)
- Store streams of records in a fault-tolerant durable way.
- Process streams of records as they occur.

Applications

- Building real-time streaming pipelines that reliably get data between systems or apps
- Building real-time streaming applications that transform or react to the streams of data

Attaching to sources

Broker: Kafka node on the cluster

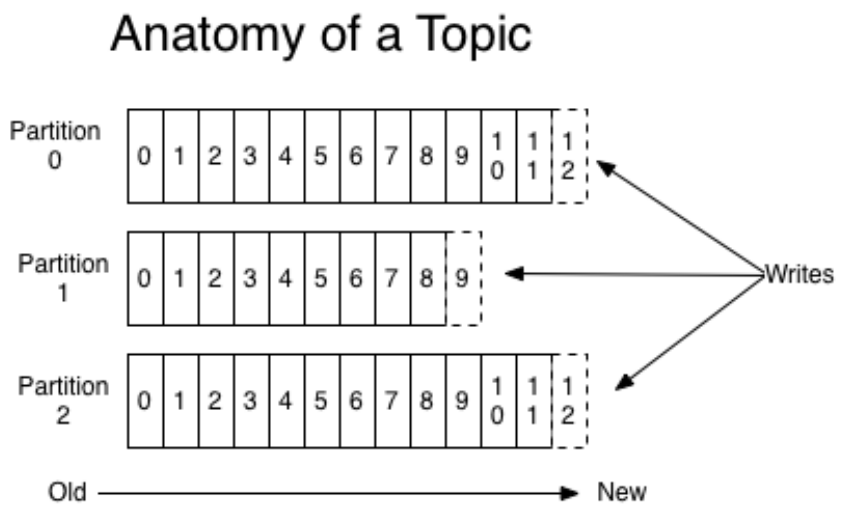
Topics: Stream of records category

- Multiple writers and readers
- Partitioned
- Replicated

Consumer-Producer pattern

- Consumer: pulls messages off of Kafka topic
- Producer: push messages into Kafka topic

Data Retention: Based on time or size



Apache ZooKeeper

Coordination of servers and services in distributed systems

- Management of distributed data
- Fault tolerant deployment
- Robust synchronization

Services provided by ZooKeeper

- Naming service for individual nodes
- Management of redundancy in high-available cluster, e.g. adding/removing nodes
- Election procedure for master nodes
- Synchronized data replication
- Monitoring and status updates of the nodes in real-time
- Management / provisioning of configuration files and deployment procedures
- Consistency models for updating replicated data sets

Optimized for Throughput in Big Data / Streaming processing

Questions ?